

Cross-Cutting Activity: Data Analysis and AI

Asena Karolin Özdemir | September 24th, 2026



Part 1: Introduction to Data Analysis

What is Data?

Data is *raw information*, before it is organized, processed or interpreted by anybody.

- Numeric → Temperature outside
- Images → Satellite observations
- Text → Survey responses
- Time series → Daily stock prices

Structured vs. Unstructured Data

- **Structured Data** has a predefined schema.
- Structured data might come in tabular, hierarchical or nested formats.

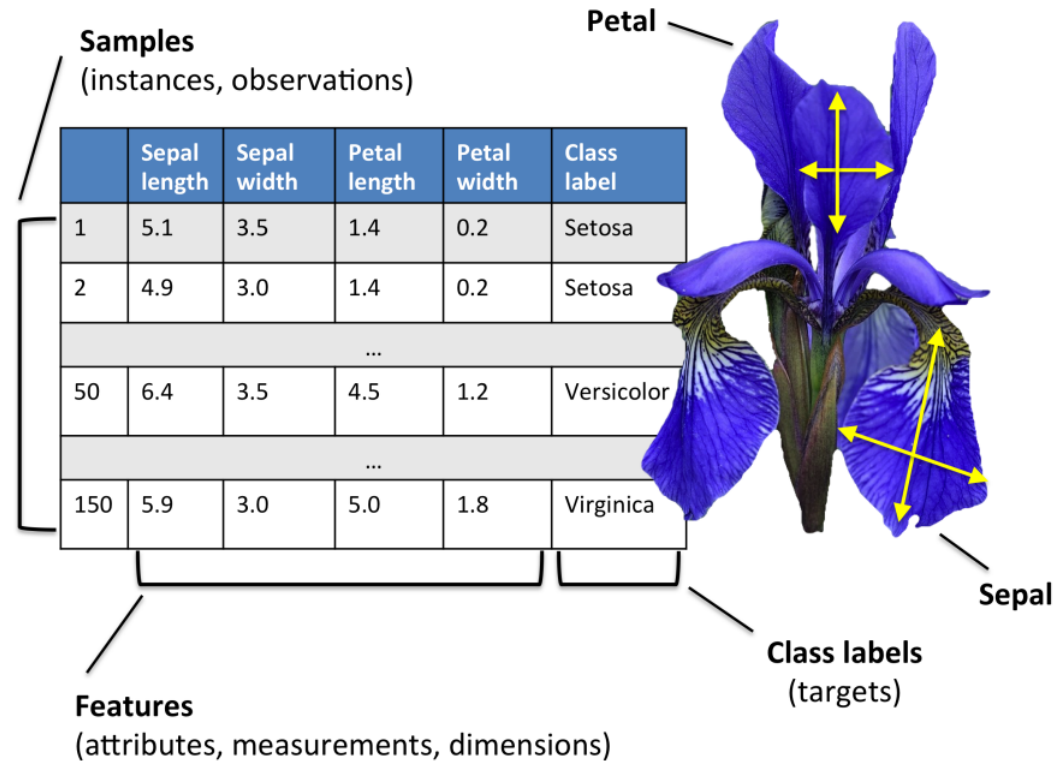


Figure: Iris Flower Dataset [3]

Structured vs. Unstructured Data

- **Unstructured Data** has no given organization or tabular schema.
- Most of the data that is available is unstructured. Images, natural language, video, audio is all unstructured.

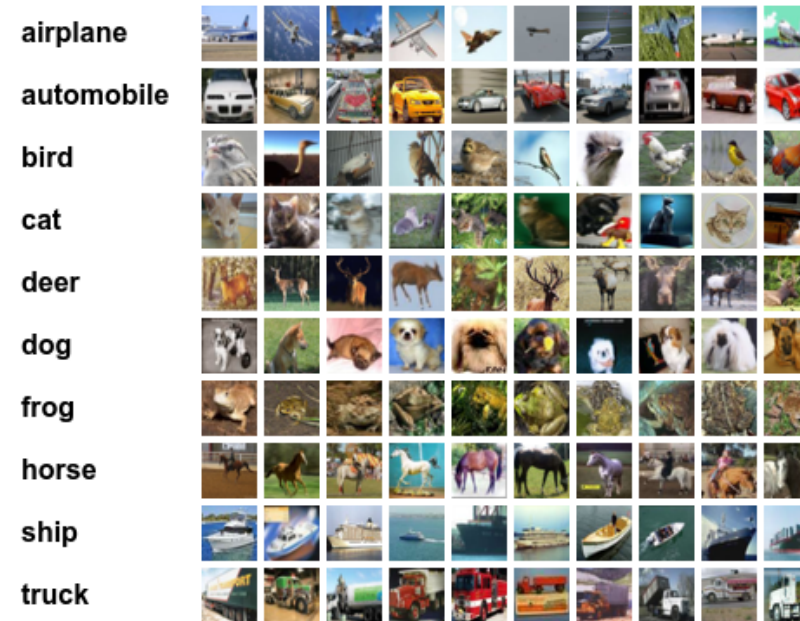


Figure: Cifar10 Image Dataset [4]

Understanding Tabular Data

- **Rows** represent **individual records**
- **Columns** represent **features**
- Features have different data types: **numerical, categorical, text, temporal...**

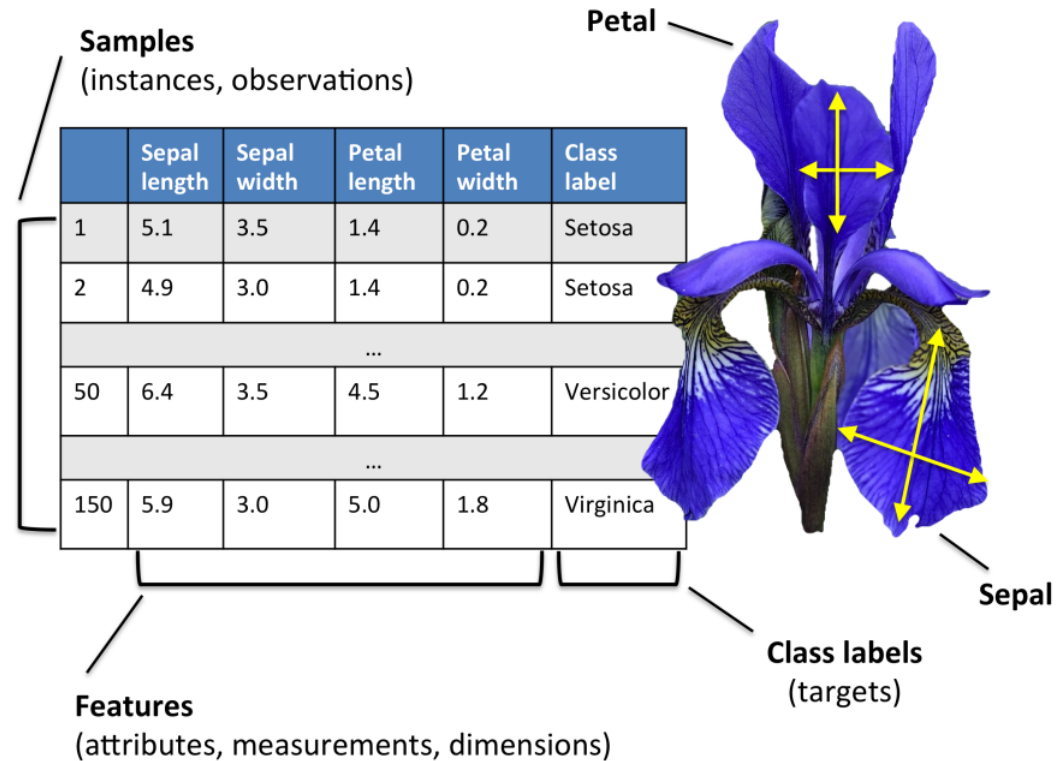


Figure: Iris Flower Dataset [3]

Understanding Tabular Data

- Data might have **missing values**.
- **Dropping** rows that have missing values → but results in the **loss of data**
- **Filling in** with either a constant, the mean, the median, the mode or the interpolation of similar data items → but **introduces artificial data** which can lead to **faulty conclusions**

Age	Sex	BMI	Diabetes	Smoking
40	M	24	Yes	Yes
45	F	25	No	No
41		24	No	
39	F	23	Yes	No
37	F	22	No	
41		22	Yes	Yes
46	F		No	Yes

Missing values

Figure: Missing Values in Tabular Data [5]

Normalizing Data

- When multiple numerical features are observed in a dataset, they may have different scales.
- e.g. the floor area of a house may be 100, while the number of rooms is 4
- However we do not want the room number to appear less important just because it is at a different scale
- Thus we normalize our data. There are many different ways to normalize, but **z-score** normalization and **min-max** normalization are among the most common ones.

■ z-score normalization

$$x' = \frac{x - \mu}{\sigma}$$

- This roughly distributes the data around 0

■ min-max normalization

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

- This puts the data into a range from 0 to 1

Understanding Unstructured Data

- **Unstructured data** can have many different forms, so we cannot generalize a way we can extract and inspect features.
- Different types of data, might require different observations.

Example: Understanding Image Data

- To a computer each image is a collection of numeric values with the dimensions of $H \times W \times C$
- H and W refer to the number of pixels that construct the image's **height and width**
- C refers to the number of **channels**. A grayscale image has only one channel, whereas a colored (RGB) image has three channels, one for each primary color.
- Images can also have missing parts, such as **blurry regions** or **corrupted pixels**.

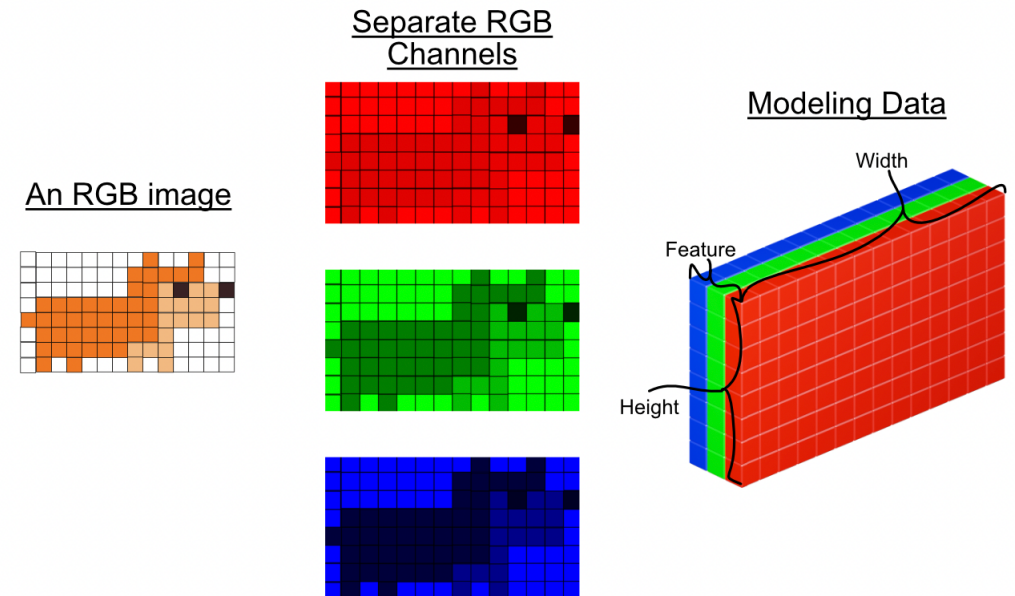


Figure: Image Data [2]

Outliers

- Datasets follow certain **distributions**
- Data items that do not fit the distribution of the rest of the data, are called **outliers**
- It is difficult to say whether an outlier is an **error in the data or a genuine extreme case**.
- Therefore one should not dismiss outliers before properly investigating them.
- Outliers are generally harder to predict.

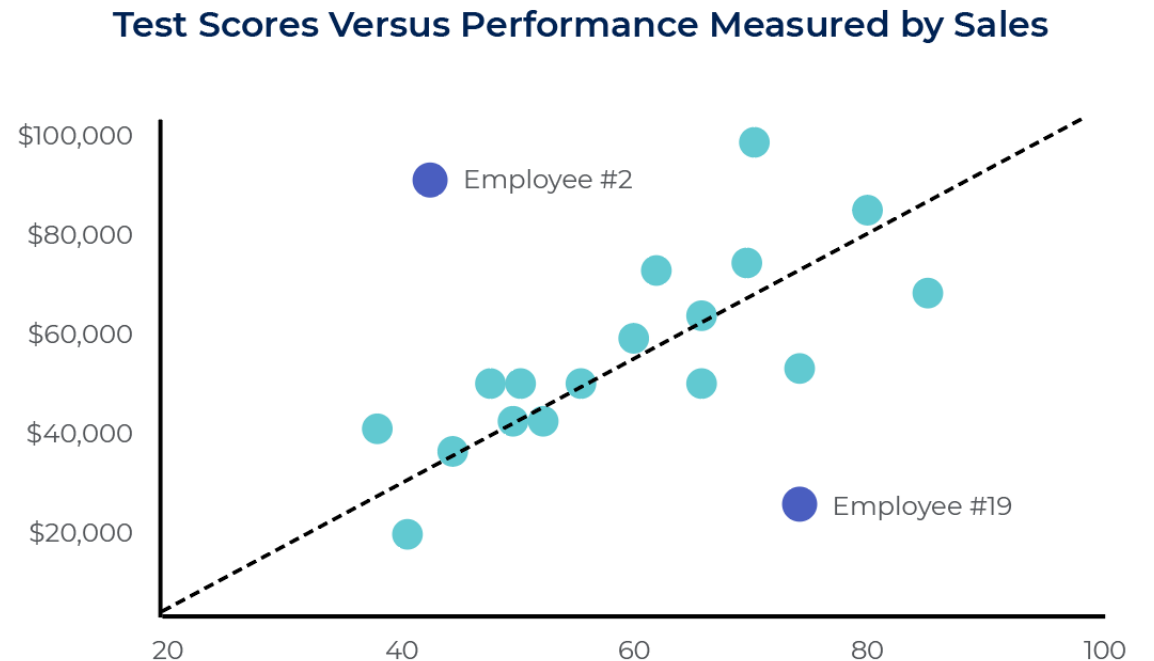


Figure: Outliers in a Dataset [7]

Data Augmentation

- Having a lot of data available is usually desired however not always possible.
- Therefore existing data is often augmented through different techniques to artificially extend the data.

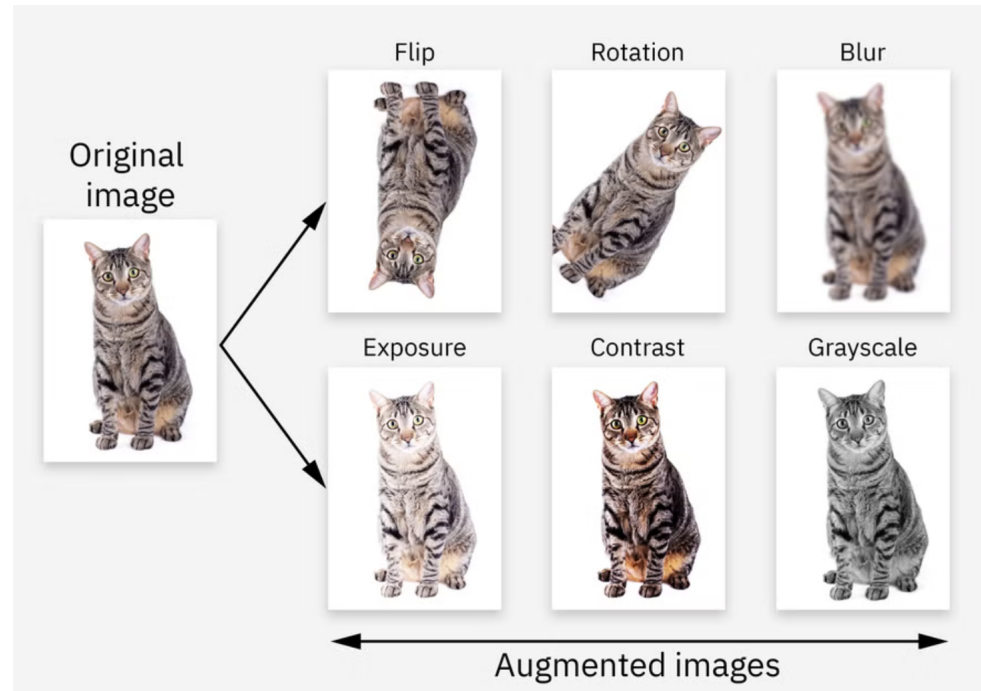


Figure: Outliers in a Dataset [8]

Part 2: From Data to Artificial Intelligence

Detour: Version Management

- Keep track of changes to code and other files
- Reproduce previous versions of your work
- Experiment without losing working versions
- Collaborate and share your work

Git → Version control system **GitHub** → Platform for hosting repositories

Edit → Commit → Push → GitHub

Commit: A snapshot of your work at a specific point in time

What is Artificial Intelligence?

Artificial intelligence (AI) is the field of building systems that perform tasks that we associate with human intelligence.

- Recognizing objects in images
- Understanding and communicating in natural languages
- Playing games
- Making predictions
- Planning and decision making

AI vs ML vs DL

- **Artificial Intelligence:** The field of creating machines that can perform tasks requiring human-like intelligence.
- **Machine Learning:** A subset of AI where machines learn patterns from data to make predictions or decisions rather than being explicitly programmed for every rule.
- **Deep Learning:** A subset of ML that uses multi-layered neural networks to learn complex patterns from data.

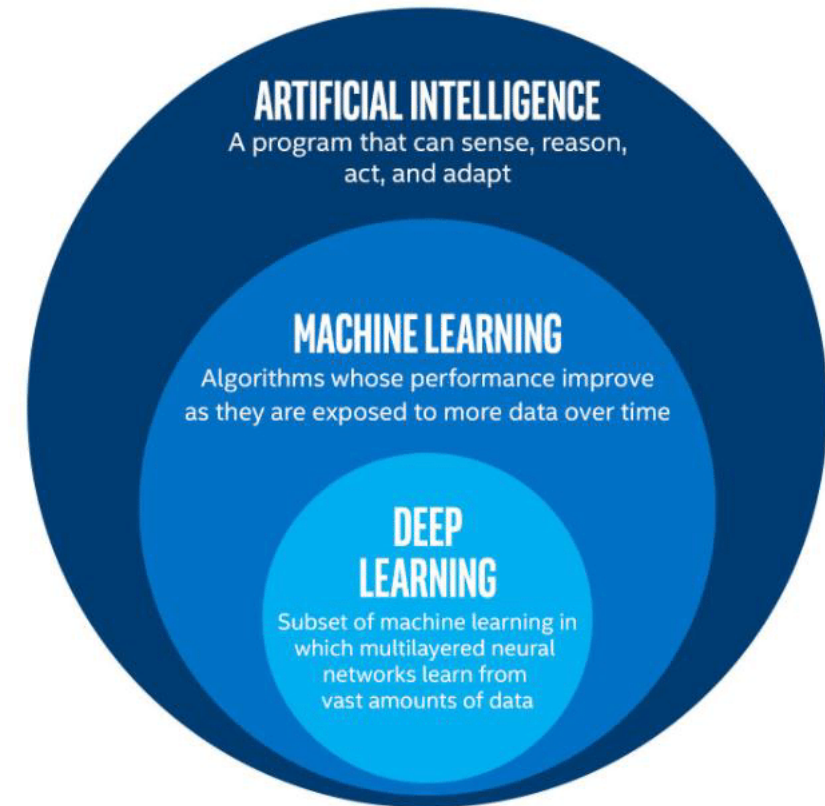


Figure: Relationship Between AI, ML and DL [1]

Types of Learning: Supervised Learning

- In **supervised learning** the model is given labeled data.
- The model learns the relationship between the data and the label and is able to predict the labels when given unseen test data.
- Typical tasks for supervised learning are **classification** (prediction of a discrete category) and **regression** (prediction of a continuous numerical value).

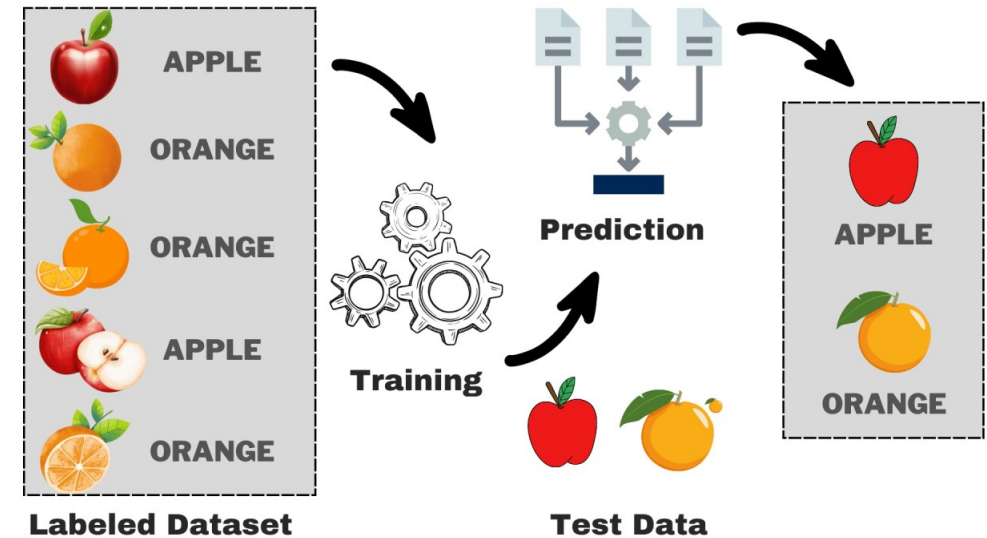


Figure: Supervised Learning Example[6]

Types of Learning: Unsupervised Learning

- In **unsupervised learning** the model is given **unlabeled data**.
- The model is trained to discover patterns, structures or groupings within the data.
- Typical tasks for unsupervised learning are **clustering**, **dimensionality reduction** and **anomaly detection**.

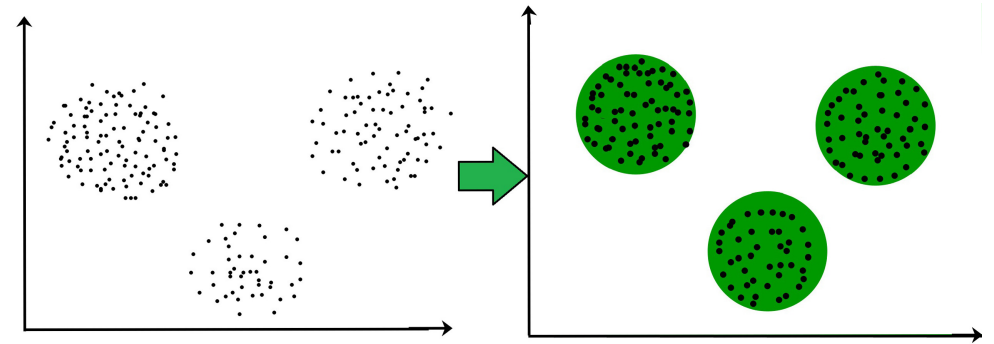


Figure: Unsupervised Learning Example[6]

Types of Learning: Self-Supervised Learning

- In **self-supervised learning** supervision is created from the data itself.
 - e.g. A word within a sentence is masked and the model is trained to predict it.
"The weather today is sunny." → "The weather today is [MASK]."
The original text can now be considered a label.
 - e.g. Encoder-Decoder Architectures (Autoencoders). An encoder model is trained to squeeze high dimensional input into a more compact low-dimensional output, while a decoder model is trained to reconstruct the original input from the output of the encoder.

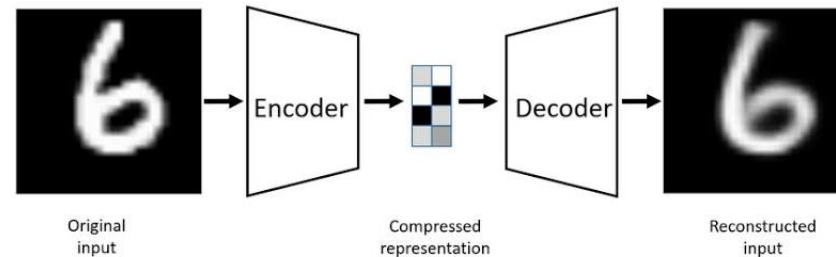


Figure: Autoencoder Architecture[6]

Further Types of Learning

- **Semi-Supervised Learning:** The model uses a small amount of labeled data together with a larger amount of unlabeled data. Used as an alternative to supervised learning when available labeled data is not sufficient.
- **Reinforcement Learning:** An agent interacts with an environment by performing actions and receiving feedback in the form of rewards or penalties. The agent learns to choose actions that maximize its cumulative reward. Most frequently used in robotics.

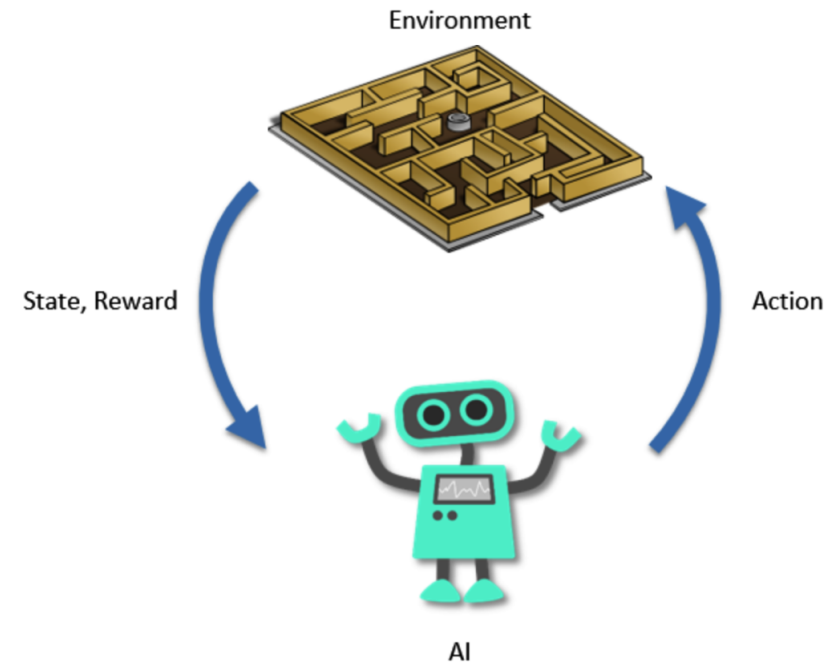
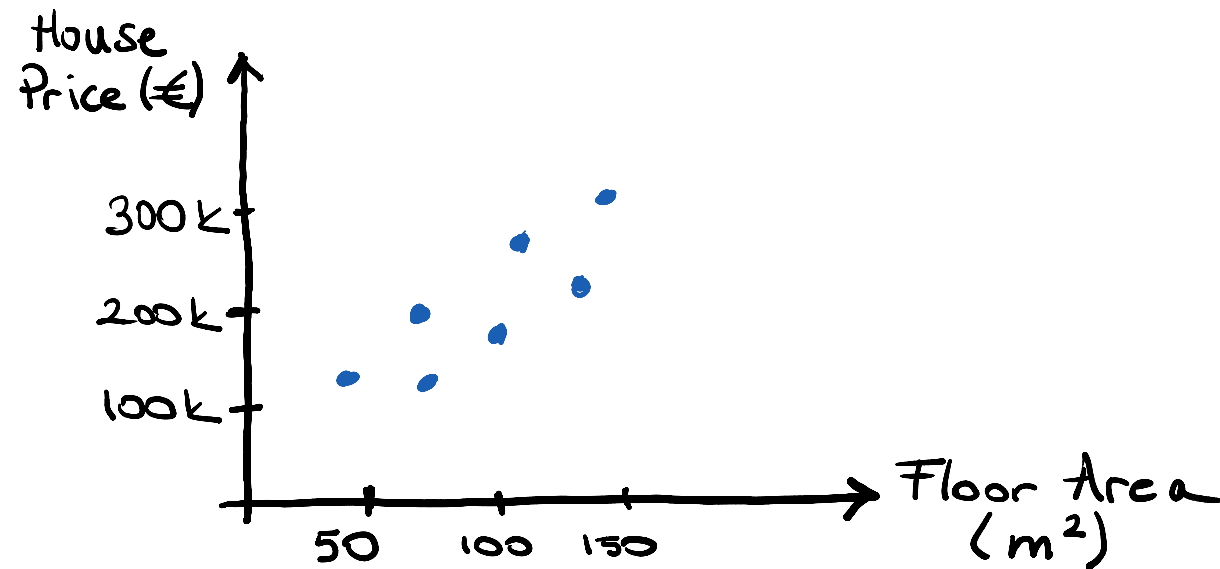


Figure: Reinforcement Learning Example[6]

Part 3: Machine Learning

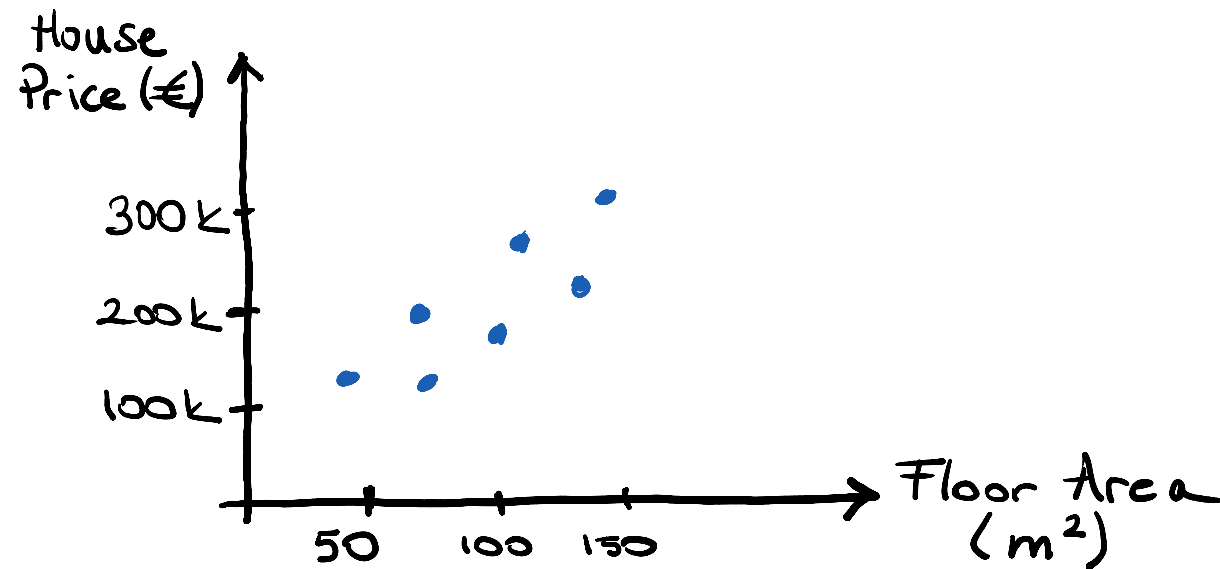
Linear Regression Example

- **Example:** Trying to predict the price of a house with 60m^2 based off of available house price data.



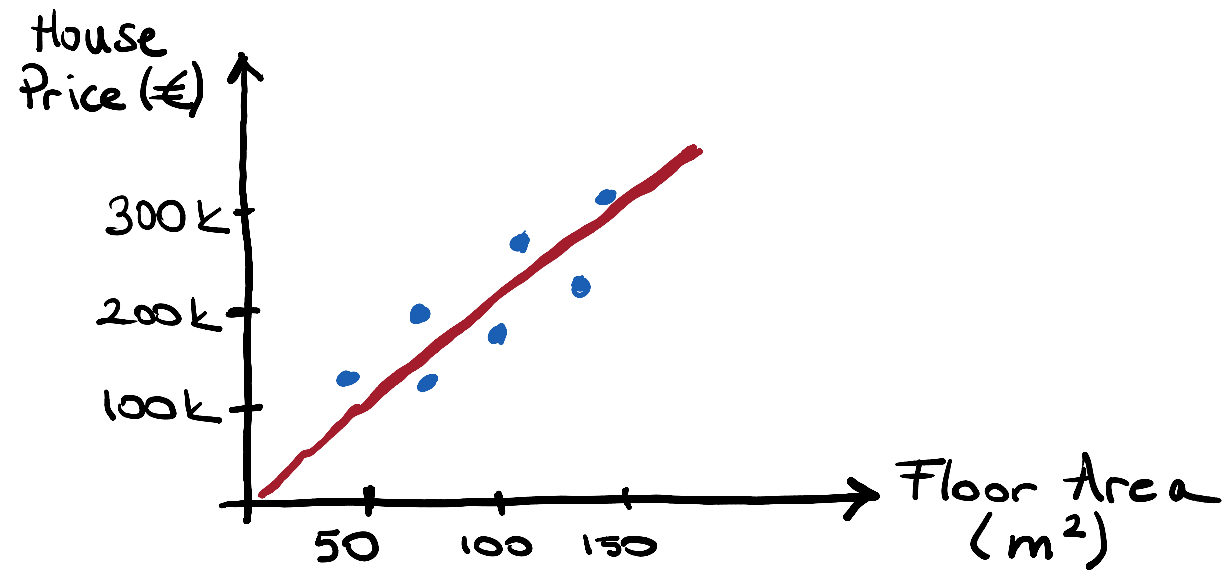
Linear Regression

- This looks like a **linear relationship**, so we could try to fit a line to this data.



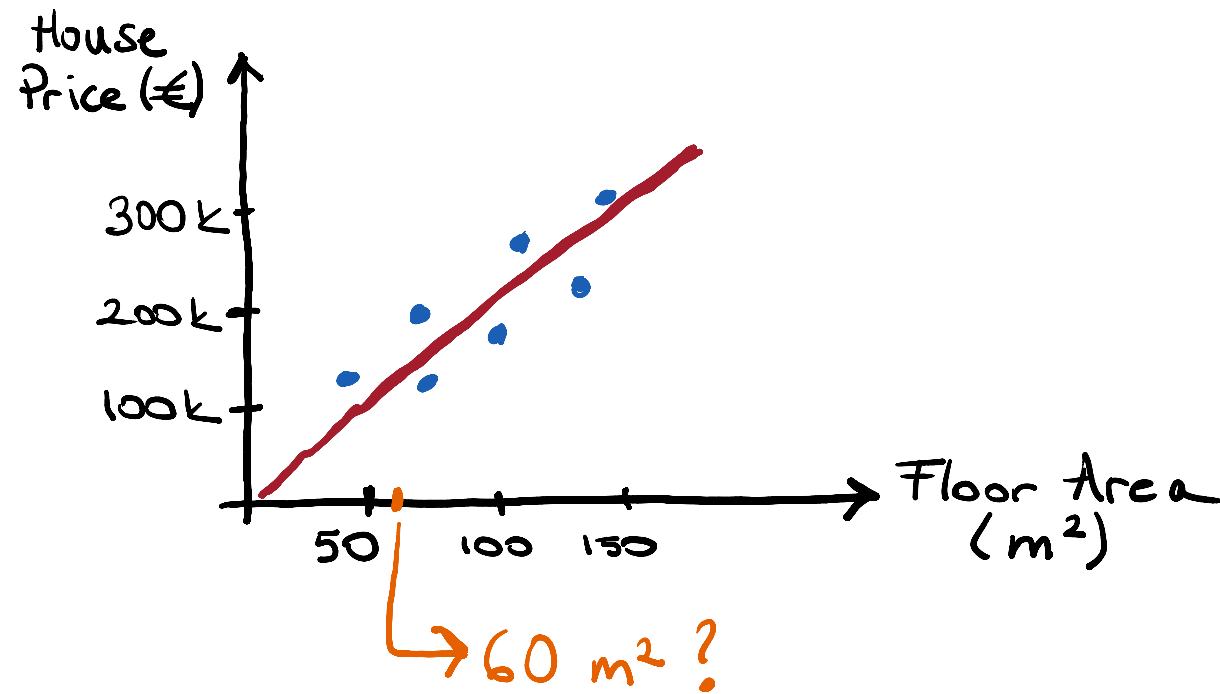
Linear Regression

- Visually this looks like a good fit.



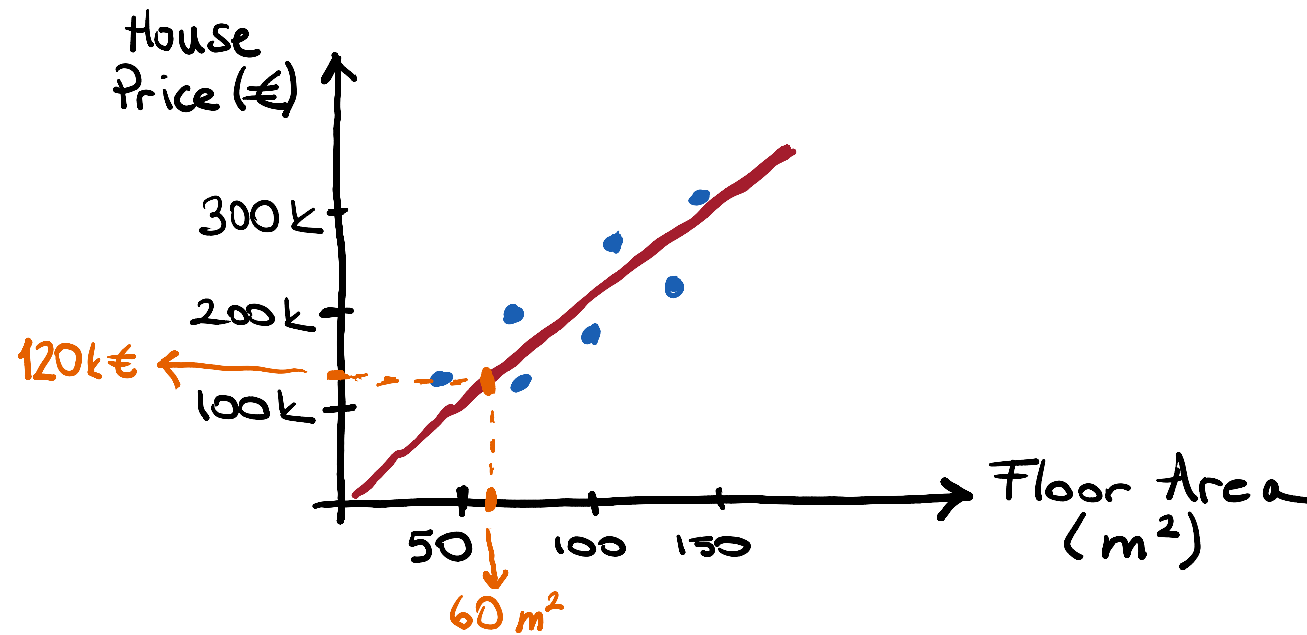
Linear Regression

- Now we just need to plug in 60m^2 into the function.



Linear Regression

- Our house should be sold for 120k Euros.
- We just did linear regression!



Linear Regression

- But how would a computer do this?
- A computer would not be able to "see" whether this is a good fit.
- Also this was a very simple example.
- In real life examples, data is more complex and larger so that humans wouldn't be able to just "see" either.
- What do we do?

Linear Regression

- We need a more systematic approach.
- We know that our line will be defined by:

$$\hat{y} = mx + b$$

where $\hat{y}$ is the predicted house price (the output),

x is the floor area (the input),

m is the slope and b is the intercept.

- m and b are the parameters that we are trying to **learn** from our data.

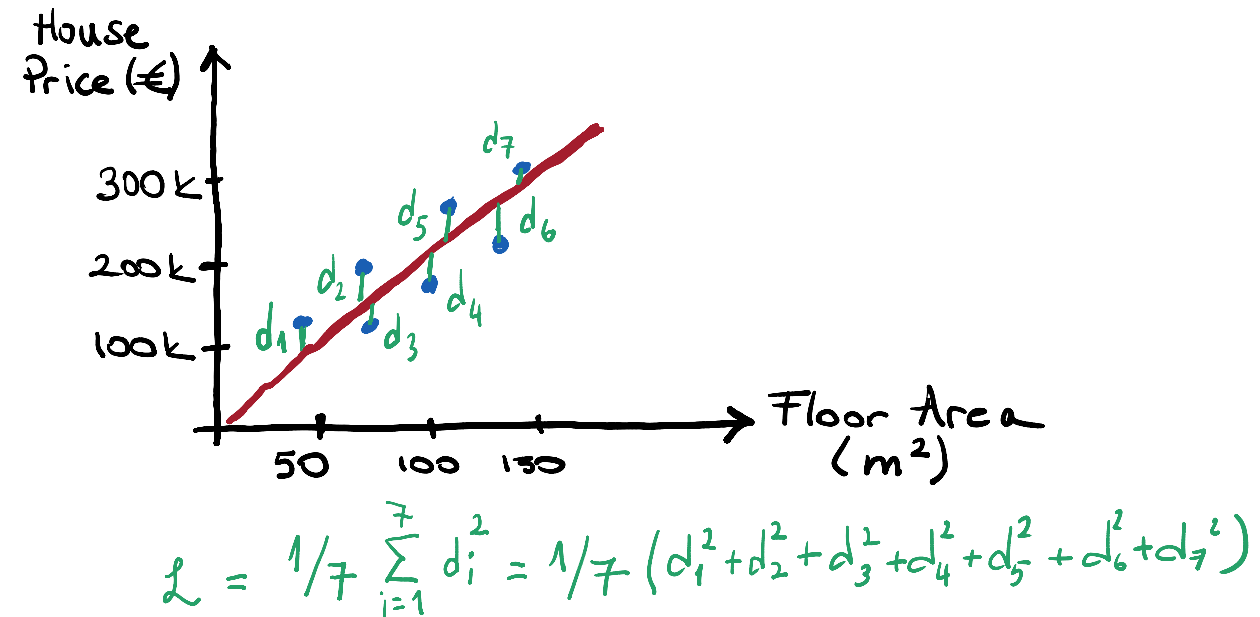
Mean Squared Error Loss

- Calculate the mean squared difference between the actual and predicted values.

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where $\hat{y}$ is the predicted house price and y is the true house price.

- This is the mse (mean squared error) loss, which is very frequently used in machine learning.
- The mse loss is a metric that tells us how good our predictions are.



Linear Regression

- Now we have a metric that tells us how good our current predictions are.
- Thus we need to minimize our loss.
- The only thing we can do is to update our parameters m and b such that $\hat{y} = mx + b$ becomes closer to the ground truth y .
- But how do we update the parameters?

Gradient Decent

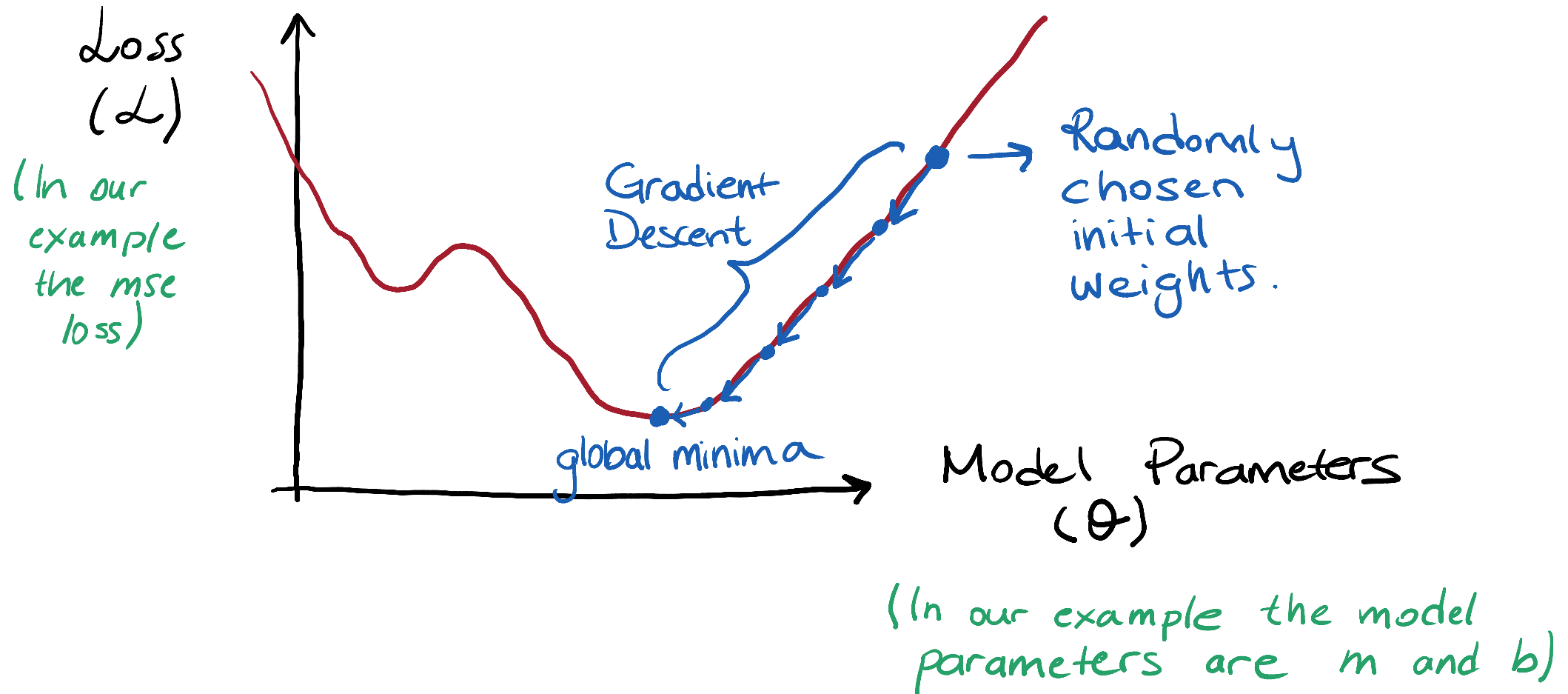
■ An optimization technique used to train machine learning models by minimizing the loss function.

1. Start with random model weights (in our case slope m and intercept b)
2. Calculate the loss function for all data points (the forward pass)
3. Calculate how much each parameter contributes to the loss (calculate the gradients)
4. Update the parameters in the direction that reduces the loss (the **learning rate** η controls the step size)

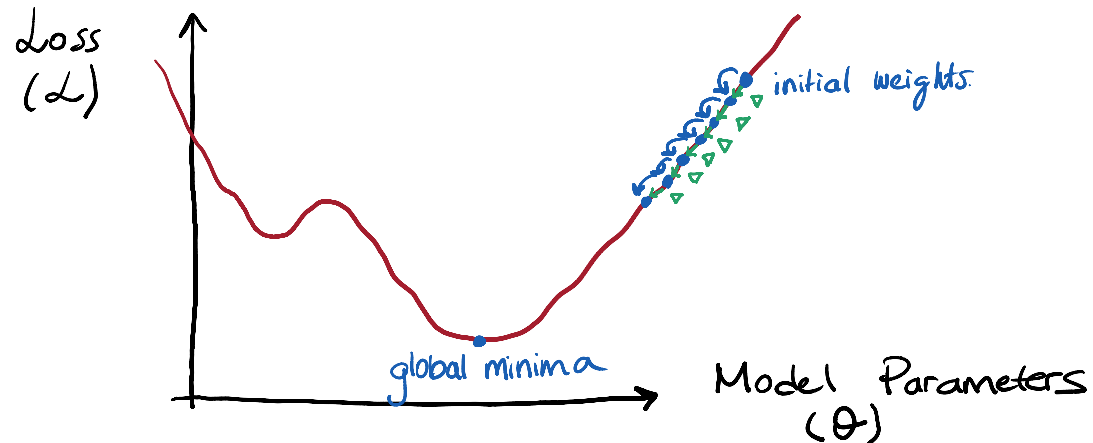
$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

5. Repeat steps 2 to 5 until error no longer changes

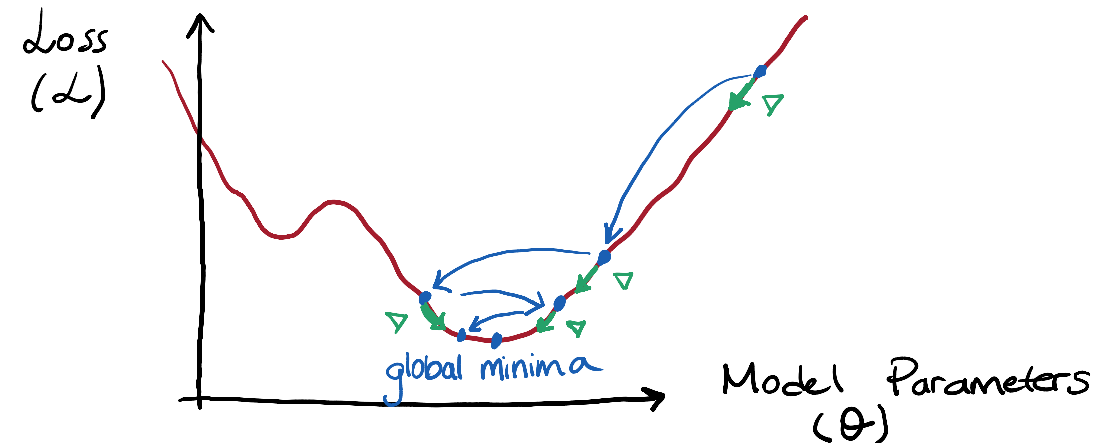
Gradient Decent



Importance of Learning Rate



Learning rate too low
→ Slow convergence



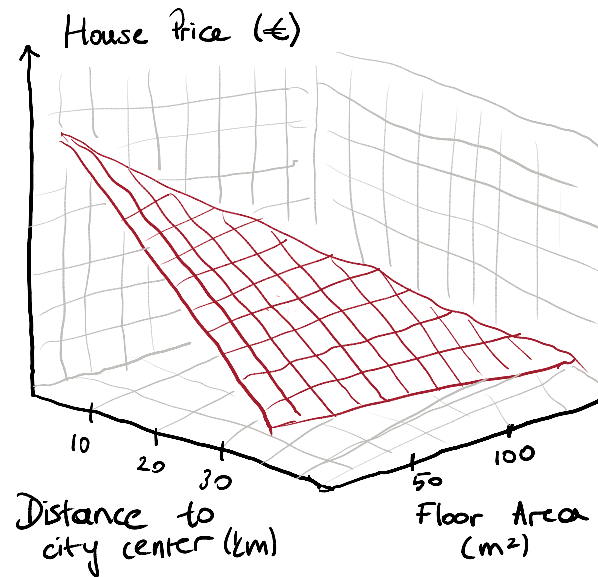
Learning rate too high
→ Oscillations around minima

When to update model parameters?

- **Batch Gradient Descent:** The loss and gradients are calculated for the **entire dataset**, and averaged, before an update to the model parameters is made. Best representation of the data, but slow.
- **Stochastic Gradient Descent:** The loss and gradients are calculated for a **single data item**, and the model parameters are updated. Fast but very affected by outliers.
- **Mini-Batch Gradient Descent:** The loss and gradients are calculated for a **subset of the dataset**, and averaged, before an update to the model parameters is made. Good middle ground.

Simple vs Multiple Linear Regression

- **Simple Linear Regression:** Fitting a line based on a single input (what we just did)
- **Multiple Linear Regression:** Predicting an output based on multiple inputs. This corresponds to fitting a plane on multiple dimensions. e.g. predicting house prices based on floor area as well as distance to city center.



Limitations and connection to the broader field of machine learning

- Linear regression can only represent linear relationships, which excludes many use cases.
- But the baseline idea of fitting a line (or more generally a curve) to the data in order to represent the relationship between input and output is essential to all machine learning approaches.
- Neural networks for example, work on the same principle.

What about classification?

- What if we have a **classification problem**?
- e.g. predicting whether an email is **spam or not spam**
- $y = 0 \rightarrow$ not spam
 $y = 1 \rightarrow$ spam
- Can we still use linear regression $\hat{y} = mx + b$?
- This approach will give us values such as 2,8 or 2026, how would we interpret them?
- It would be more practical to have values between 0 and 1 so that we could interpret them as probabilities.

Sigmoid Function

- In order to transform the output of a linear relationship between 0 and 1 we pass it through the **sigmoid function** (σ) which is defined by:

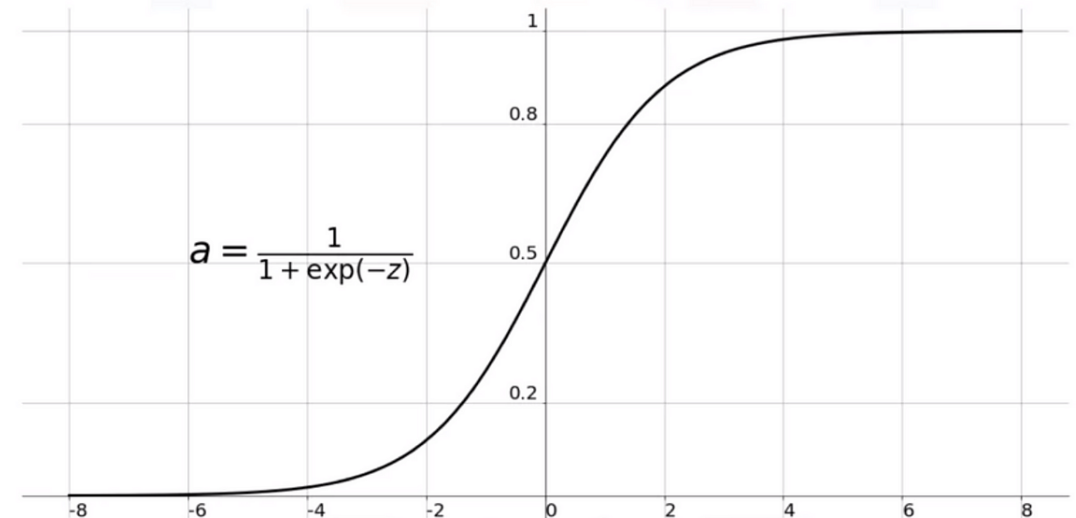
$$y = \frac{1}{1 + e^{-z}}$$

- So our model gets updated in the following way:

$$z = mx + b$$

$$\hat{y} = \sigma(z)$$

Sigmoid Function



Binary Cross-Entropy Loss

- But what do we do about the loss function?
- If the true class $y = 1$, then we would like to reward predictions close to 1 and punish predictions close to 0. Thus

$$\mathcal{L} = -\log(\hat{y})$$

works as a good loss function.

- However if the true class $y = 0$, it is the opposite.

$$\mathcal{L} = -\log(1 - \hat{y})$$

- Combining the two, we get:

$$\mathcal{L} = -y\log(\hat{y}) - (1 - y)\log(1 - \hat{y})$$

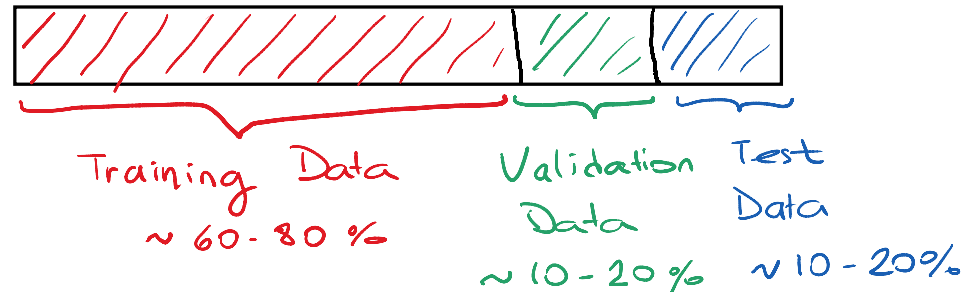
- This is the **binary cross-entropy loss**.
- Depending on the true class, one of the terms disappears.

Logistic Regression

- Now we have all the ingredients ready for classification with logistic regression.
- The data with inputs x and outputs y
- The model $\hat{y} = \sigma(mx + b)$ with trainable parameters m and b
- The loss function $\mathcal{L} = -y\log(\hat{y}) - (1 - y)\log(1 - \hat{y})$
- The ability to do gradient descent to train m and b

How to split data for machine learning

- **Training data:** The data that is used to learn our model parameters from.
- **Validation data:** The data reserved to make high level decisions about our training that is separate from our training data.
- **Test data:** The data to test the performance of our fully trained model.
- We need to split our dataset into these three categories before using it.
- It is important that the split happens after the data is randomized so that the distribution of the data is the same for all three dataset splits.



More Generally..

- In order to train any supervised machine learning model we need the following:
- Training data: inputs x and corresponding outputs y
- A model with trainable parameters θ , i.e. an assumed function $\hat{y} = f(x; \theta)$
- A suitable loss function to measure how well our prediction $\hat{y}$ matches the true y
- An optimization algorithm to adjust θ and minimize the loss

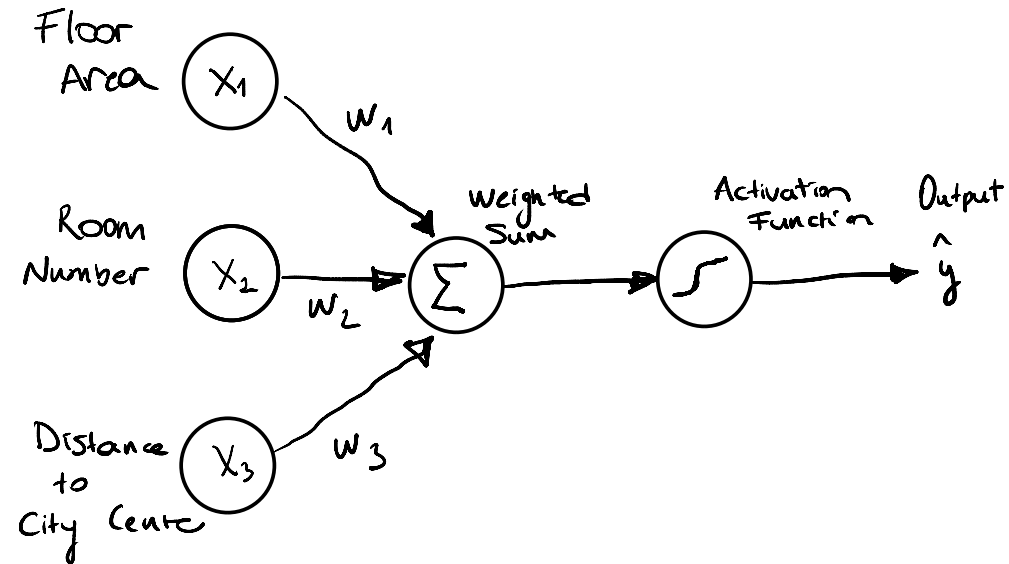
Linear vs. Logistic Regression

	Linear Regression	Logistic Regression
Goal	Regression	Binary Classification
Training data	$y \in \mathbb{R}$	$y \in \{0, 1\}$
Model	$\hat{y} = wx + b$	$\hat{y} = \sigma(wx + b)$
Loss function	Mean Squared Error	Binary Cross-Entropy
Optimization	e.g. Gradient Descent	e.g. Gradient Descent

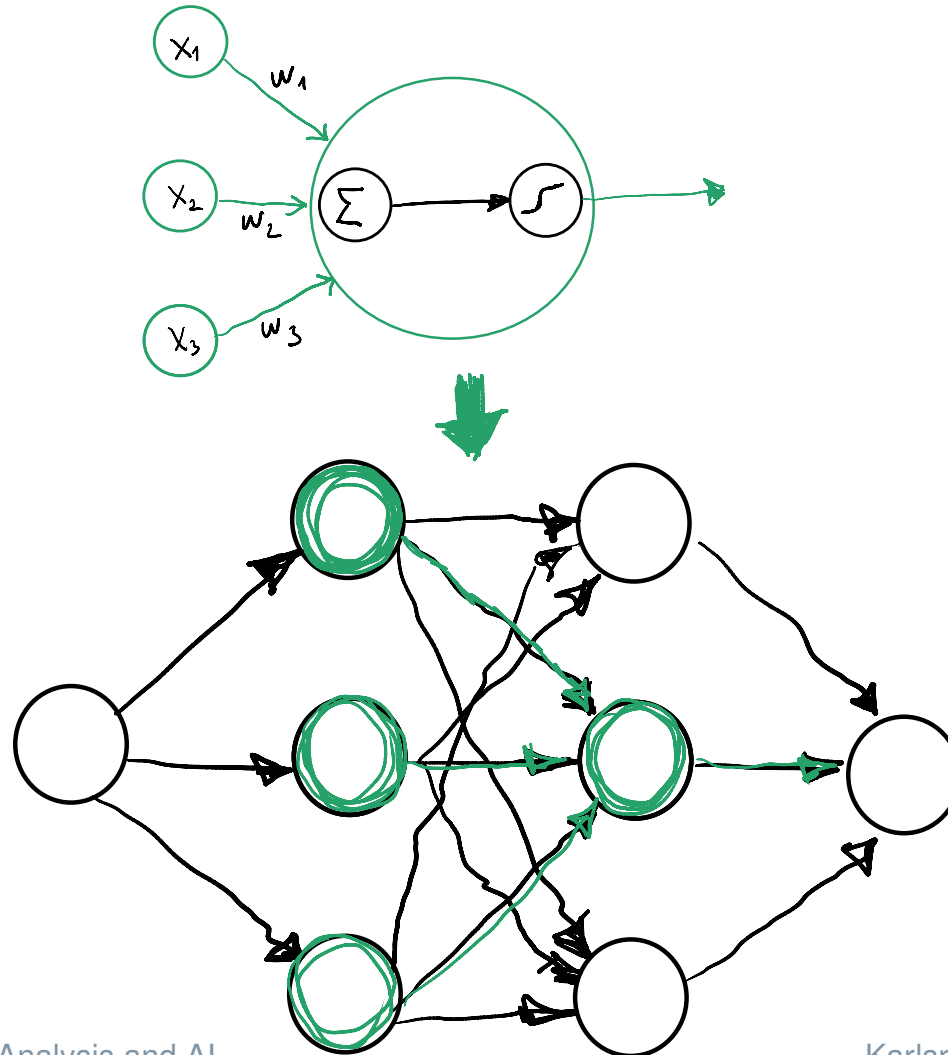
What about Neural Networks?

- The same general recipe applies!
- We still have:
 - Training data (x, y)
 - A model with trainable parameters
 - A loss function
 - An optimization algorithm
- The main difference:
 - The model can be much more complex
 - Many more trainable parameters
 - Can learn **nonlinear relationships**

Single Layer Perceptron



Multi Layer Perceptron



Why do we need activation functions?

- Linear relations are not enough to model all problems.
- But what if we use multiple layers of linear relations?

$$y' = W_2(W_1x + b_1) + b_2$$

- But this reduces to a linear relation nonetheless

$$y' = W_2W_1x + W_2b_1 + b_2$$

$$y' = W'x = b'$$

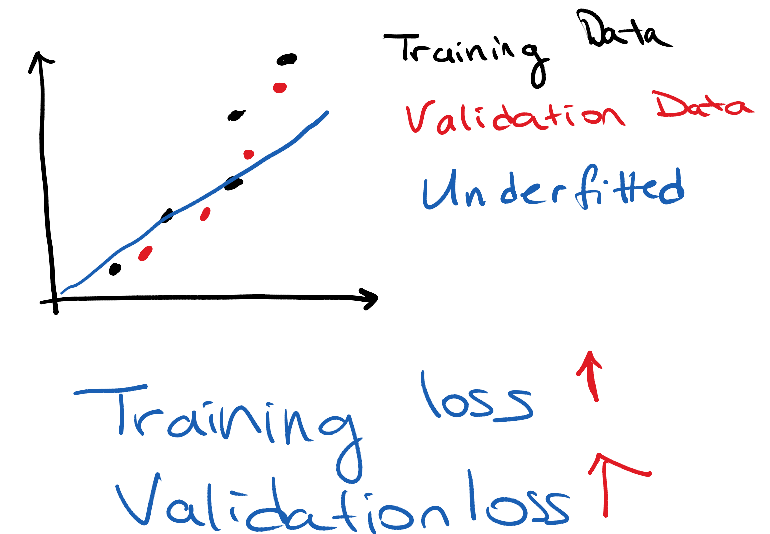
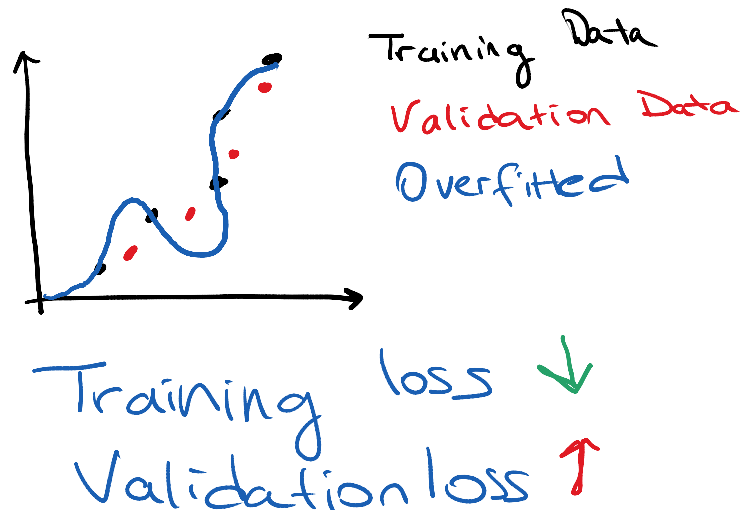
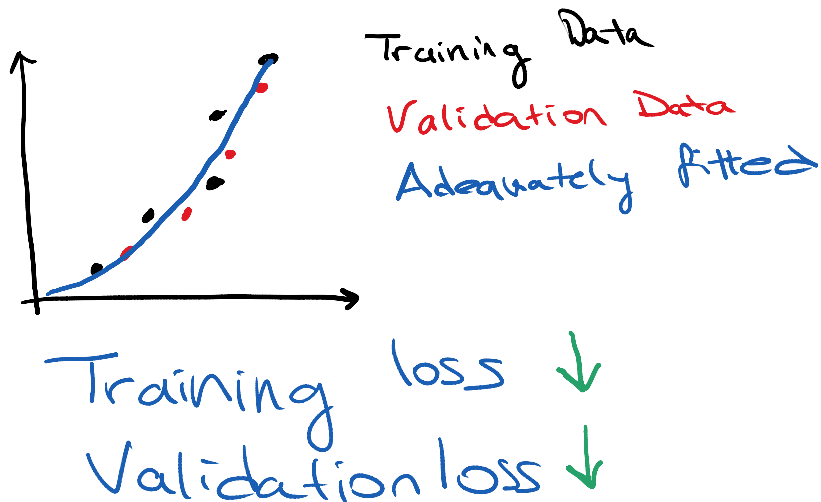
where

$$W' = W_2W_1 \text{ and } b' = W_2b_1 + b_2$$

- Therefore we need an activation function to break the linearity.

Overfitting and Underfitting

- During training our goal is to adequately fit our training data but still keep our function generalizable to validation and test data.
- **Overfitting:** When our model fits the training very well but loses generalization quality.
- **Underfitting:** When our training model cannot fit the training data nor the validation data.



Overfitting and Underfitting

- **Underfitting** is usually caused by an **insufficiently complex model** or **insufficient training**.
- Potential solutions are reconsidering the model architecture or continuing training.
- **Overfitting** on the other hand, is caused by a **overly complex model** and **too much training**.
- Potential solutions are **early stopping**, **dropout** or **weight decay**.

Hyper-parameter Tuning

- The weights and biases of the model are learned during training but there are other parameters that influence the model as well such as:
 - The number of epochs we train for
 - The batch size
 - The learning rate
 - Number of layers of our model and the size of each layer.
- These are called hyper-parameters and need to be tuned outside of the training loop.
- There is no guaranteed way to find the best set of hyper-parameters, therefore hyperparameter tuning is still an open research field.

What we didn't covered today

- CNNs
- Transformers
- Attention
- LSTMs/RNNs
- LLMs
- VAEs
- Bias in data
- ...
- So there is more to learn!

Further Resources

- There is a lot of material on this subject available online, so choose your own favorite but here are my picks for you.
- For conceptual understanding and motivation on neural networks check out 3b1b videos on youtube:
`https://www.youtube.com/playlist?list=PLZHQ0b0WTQDNU6R1\_67000Dx\_ZCJB-3pi`
- For hands on programming tutorials check out Pytorch's tutorials:
`https://docs.pytorch.org/tutorials/beginner/basics/intro.html`
- Also the code we implemented is available at: `https://github.com/RAI-SCC/hyperion\_school\_data\_ai`

Image Sources I

- [1] Arbaj Ansari et al. “Handwritten text recognition using deep learning algorithms”. In: *2022 4th International Conference on Artificial Intelligence and Speech Technology (AIST)*. IEEE. 2022, pp. 1–6.
- [2] *Convolutional Networks - Intuitively and Exhaustively Explained*.
<https://towardsdatascience.com/convolutional-networks-intuitively-and-exhaustively-explained-ab08f6353f96/>. Accessed: 2026-08-28.
- [3] *Exploratory Data Analysis : Iris Dataset*.
<https://medium.com/analytics-vidhya/exploratory-data-analysis-iris-dataset-4df6f045cda>.
Accessed: 2026-08-27.

Image Sources II

- [4] *Image Classification on CIFAR-10 with 15 Lines of Code*. <https://medium.com/@sanskarhasija/image-classification-on-cifar-10-with-15-lines-of-code-52341e7fb00c>. Accessed: 2026-08-27.
- [5] Ahmed M Salih. “Common Steps in Machine Learning Might Hinder The Explainability Aims in Medicine”. In: *arXiv preprint arXiv:2409.00155* (2024).
- [6] *Understanding Supervised Learning: Theory and Overview*.
<https://www.kdnuggets.com/understanding-supervised-learning-theory-and-overview>. Accessed: 2026-08-31.
- [7] *What is an outlier?* <https://www.criteriacorp.com/resources/glossary/outlier>. Accessed: 2026-08-28.

Image Sources III

- [8] *What is data augmentation?* <https://www.ibm.com/think/topics/data-augmentation>. Accessed: 2026-09-01.